

算法选择与性能分析答辩文档

1. 算法总览

本系统围绕“景区/校园推荐、路线规划、场所查询、旅游日记、美食推荐、AIGC 动画生成”六类核心功能设计数据结构和算法。整体思路是：查询类功能优先使用索引和模糊匹配，推荐类功能使用 Top-K 排序，路线类功能使用图结构和最短路径算法，文本内容使用全文检索和无损压缩。

功能模块	使用算法/数据结构	代码位置	选择原因
旅游推荐	Top-K 排序、最小堆、加权评分	backend/app/services/sort_service.py	用户通常只看前 10 条，不需要完整排序
景区/校园查询	Trie 字典树、编辑距离模糊匹配、结果排序	backend/app/services/search_service.py	支持名称、类别、关键词查询，兼顾效率和容错
单点路线规划	图邻接表、Dijkstra、最小堆	backend/app/services/graph_service.py	边权非负，适合求最短距离和最短时间
多点路线规划	Dijkstra 距离矩阵、贪心 TSP 近似、2-opt 优化	backend/app/services/graph_service.py	多景点访问属于 TSP 问题，精确解成本高，近似算法更适合演示系统
交通工具策略	边过滤、速度修正、拥挤度权重	backend/app/services/graph_service.py	支持步行、自行车、电瓶车和混合通行
室内导航	室内图、Dijkstra	backend/app/services/graph_service.py	教学楼/博物馆内部同样可抽象为节点和边
场所查询	Dijkstra 实际路径距离排序	backend/app/services/sort_service.py	按真实道路距离排序，而不是直线距离
旅游日记推荐	Top-K 排序、热度/评分/兴趣加权	backend/app/services/sort_service.py	支持按热度、评分和个人兴趣推荐
日记标题查询	Trie/模糊查找	backend/app/routes/diary.py	面向大量动态日记，支持高效名称查询
日记全文检索	Whoosh + BM25F	backend/app/services/search_service.py	适合长文本检索，按相关性排序
日记压缩存储	zlib 无损压缩	backend/app/models/diary.py	保存原文内容不丢失，同时降低存储占用
美食推荐	Top-K 排序、距离评分、菜系过滤	backend/app/services/sort_service.py	满足热度、评价、距离、菜系等多条件推荐
美食搜索	编辑距离模糊查找、结果排序	backend/app/services/search_service.py	支持美食名、菜系、饭店名等内容查询

功能模块	使用算法/数据结构	代码位置	选择原因
AIGC 动画	图生视频接口调用、参数化生成	backend/app/routes/diary.py	视频生成属于模型能力，系统负责图片、提示词和参数管理

2. 自定义数据结构

2.1 图结构 Graph

系统把景区、校园和建筑室内空间统一抽象为图：

- 节点表示景点、建筑、服务设施、路口、楼层、电梯、房间等。
- 边表示道路、楼道、电梯连接等。
- 每条边保存距离、道路类型、拥挤度、可用交通工具等信息。

实现方式为邻接表：

```
node_id -> [(neighbor_id, distance, time_cost, edge_info), ...]
```

选择原因：

- 邻接表适合稀疏图，景区道路图通常不是完全图。
- 遍历某个节点的相邻道路效率高。
- 便于 Dijkstra 算法按边扩展。

复杂度：

- 存储复杂度： $O(V + E)$
- 查询某节点邻居： $O(\deg(V))$

2.2 最小堆 MinHeap

系统自实现了最小堆，主要用于 Dijkstra 的优先队列，以及 Top-K 排序中的候选维护。

选择原因：

- Dijkstra 每次需要取当前距离最小的未确定节点。
- 使用堆后，不需要每轮线性扫描所有节点。

复杂度：

- 插入： $O(\log N)$
- 弹出最小值： $O(\log N)$
- 查看堆顶： $O(1)$

2.3 Top-K 最小堆 MinHeapForTopK

推荐场景中，系统不对所有结果完整排序，而是维护一个容量为 K 的最小堆。

核心思路：

1. 堆中始终保存当前分数最高的 K 个元素。
2. 堆顶是这 K 个元素中分数最低的。
3. 新元素如果比堆顶分数高，就替换堆顶。

选择原因：

- 课程要求强调“用户通常只看前 10 个，不经过完全排序可以排好前 10 个”。
- 当数据动态变化时，只需要重新计算候选分数并维护小堆。
- 相比完整排序，Top-K 更节省计算量。

复杂度：

- 完整排序： $O(N \log N)$
- Top-K 排序： $O(N \log K)$
- 当 $K=10$ 时，效率更高。

2.4 Trie 字典树

系统实现了 TrieNode 和 Trie，用于名称、类别等字段的前缀查找。

选择原因：

- 前缀匹配效率高。
- 适合景区名、校园名、日记标题等字符串查询。
- 每个前缀节点保存对应数据 ID，查询时可以快速得到候选集合。

复杂度：

- 插入： $O(L)$
- 前缀查询： $O(L)$
- L 为查询字符串长度。

3. 旅游推荐算法

3.1 推荐分数设计

景区和校园推荐综合考虑：

- 热度 popularity
- 评分 rating
- 用户兴趣 user_interests
- 标签 tags

综合推荐分数：

$$\text{score} = 0.4 * \text{热度分} + 0.3 * \text{评分分} + 0.3 * \text{兴趣匹配分}$$

如果用户选择按热度排序，则直接使用热度作为分数；如果选择按评分排序，则评分作为主指标，热度作为辅助指标。

3.2 为什么使用 Top-K

旅游推荐页通常只展示第一页或者前 10 个目的地。如果对 200 多个景区/校园完全排序，虽然数据量不大也能跑，但算法设计上不够贴合课程要求。Top-K 可以直接得到前 K 个推荐结果，满足“无需完全排序”的要求。

复杂度：

N 为候选景区/校园数量，K 为返回数量

时间复杂度： $O(N \log K)$

空间复杂度： $O(K)$

答辩说法：

“推荐模块没有直接调用数据库 order by 完成所有排序，而是在业务层计算热度、评分、兴趣匹配的综合分数，再用最小堆维护前 K 个结果，这样符合用户只看前十条的场景，也满足课程要求的 Top-K 排序。”

4. 查询和搜索算法

4.1 模糊查找

模糊搜索使用编辑距离，也就是 Levenshtein Distance。它通过动态规划计算两个字符串之间最少需要多少次插入、删除、替换才能相同。

动态规划状态：

$dp[i][j]$ = query 前 i 个字符 和 target 前 j 个字符 的最小编辑距离

选择原因：

- 用户输入可能不完整或有错字。
- 美食、景区、设施名称都适合使用模糊匹配提高容错率。
- 对中文短词，系统增加了精确包含匹配，避免编辑距离造成过多无关召回。

复杂度：

m 为查询词长度，n 为候选词长度

单次匹配时间复杂度： $O(mn)$

空间复杂度： $O(mn)$

4.2 Trie 前缀查找

Trie 用于景区/校园名称、分类、日记标题等前缀查询。

选择原因：

- 对“清”“清华”“故宫”等输入可以快速定位候选。
- 查询复杂度只和关键词长度有关。
- 适合名称类字段。

4.3 全文检索 BM25F

旅游日记正文较长，不能只靠标题或简单包含判断。系统使用 Whoosh 建立全文索引，并采用 BM25F 评分。

选择原因：

- 支持标题、正文、目的地、标签多个字段同时检索。
- BM25F 会考虑词频、文档长度和字段权重，比普通 contains 更适合正文搜索。
- 适合“按日记内容进行全文检索”的课程要求。

答辩说法：

“名称查询使用 Trie 和模糊查找，正文检索使用全文索引。这样把短文本查找和长文本检索分开处理，避免所有场景都用低效的全表字符串扫描。”

5. 路线规划算法

5.1 单点最短路径 Dijkstra

景区/校园道路图中，节点是地点，边是道路。边权可以是距离，也可以是时间。

最短距离策略：

`weight = distance`

最短时间策略：

真实速度 = 基础速度 * 拥挤度修正系数
`time_cost = distance / 真实速度`

系统中基础速度：

- 步行：5 km/h
- 自行车：15 km/h
- 电瓶车：20 km/h

选择 Dijkstra 的原因：

- 道路距离和时间都为非负权重。
- Dijkstra 可以稳定求出单源最短路径。
- 使用最小堆优化后，适合景区校园这种中等规模道路图。

复杂度：

V 为节点数，E 为边数
使用堆优化： $O((V + E) \log V)$
空间复杂度： $O(V + E)$

5.2 多点路线规划

多个目标点路线规划本质接近旅行商问题 TSP。TSP 精确解复杂度很高，不适合在演示系统中暴力枚举所有排列。

系统采用：

1. 对起点和所有目标点运行 Dijkstra，得到关键点之间的最短距离矩阵。
2. 用贪心算法构造初始访问顺序，每次选择当前最近的未访问目标。
3. 使用 2-opt 优化，尝试反转部分访问顺序，降低总路径成本。
4. 如果勾选返回起点，最后再规划回到起点的路径。

选择原因：

- 比暴力枚举更快。
- 路径质量比单纯贪心更好。
- 可以在前端快速响应，适合课堂演示。

复杂度说明：

k 为目标点数量

距离矩阵：需要对 k+1 个关键点运行 Dijkstra

约为 $O(k * (V + E) \log V)$

2-opt 优化：约 $O(k^2 * iter)$

答辩说法：

“多点路线不是简单把用户选择顺序串起来，而是先计算关键点之间的最短路径，再用贪心和 2-opt 找一个近似更优的访问顺序。因为多点最优路径属于 TSP 类问题，精确求解成本高，所以这里选择近似算法平衡效果和性能。”

5.3 交通工具和拥挤度策略

系统支持：

- 校园：步行、自行车
- 景区：步行、电瓶车
- 最短距离、最短时间
- 不同道路拥挤度
- 部分路段混合交通

实现方式：

- 每条边保存 transport_allowed。
- 构建图时根据交通工具过滤边。
- 自行车/电瓶车遇到只允许步行的路段，会按步行速度处理。
- 时间权重根据拥挤度修正。

选择原因：

- 贴近真实场景，不同交通工具不能走所有道路。
- 混合交通可以避免“自行车模式因为局部步行路段断开而完全失败”。
- 拥挤度使“最短时间”和“最短距离”可能产生不同路线，便于演示算法效果。

5.4 室内导航

室内导航将建筑内部抽象为图：

- 大门、楼梯、电梯、走廊、房间作为节点。
- 走廊、电梯连接作为边。
- 楼层间通过电梯边连接。

算法仍然使用 Dijkstra。

选择原因：

- 室内导航和室外道路导航本质一致，都是图上的最短路径问题。
- 复用图算法服务，架构更统一。

6. 场所查询算法

场所查询要求“不能根据直线距离”，所以系统采用实际路径距离排序。

流程：

1. 用户选择当前位置节点。
2. 根据景区/校园 ID 构建道路图。
3. 对当前位置运行一次 Dijkstra，得到到所有节点的最短路径距离。
4. 将设施节点映射到对应图节点。
5. 按最短路径距离排序并按范围过滤。

选择原因：

- 一次 Dijkstra 可以得到到所有设施的路径距离。
- 比对每个设施分别跑一次最短路径更高效。
- 结果符合实际道路可达距离，而不是地图上的直线距离。

复杂度：

一次 Dijkstra: $O((V + E) \log V)$

设施排序: $O(M \log M)$, M 为设施数量

答辩说法：

“附近设施不是按经纬度直线距离算的，而是先在道路图上计算从当前位置到各设施节点的最短路径距离，再按这个距离排序，所以更符合真实步行体验。”

7. 旅游日记算法

7.1 日记推荐

日记推荐综合考虑：

- 浏览量 `view_count`
- 平均评分 `average_rating`

- 用户兴趣标签

同样使用 Top-K 排序。

选择原因：

- 日记数量可能动态增长。
- 用户浏览时只需要优先展示最相关的前几篇。
- 热度和评分都可以动态变化，业务层实时计算分数更灵活。

7.2 日记精确查询和全文搜索

日记名称查询适合使用 Trie 或模糊查找；日记正文检索使用 Whoosh + BM25F。

选择原因：

- 标题短，适合前缀和模糊匹配。
- 正文长，适合全文索引。
- 两类查询分开处理，性能和准确性更好。

7.3 zlib 无损压缩

日记正文存入数据库前使用 zlib level=9 压缩，读取时再解压。

选择原因：

- 日记正文是文本，重复词较多，压缩收益明显。
- zlib 是无损压缩，能完整恢复用户原文。
- 数据库中存储二进制压缩内容，减少存储成本。

答辩说法：

“日记正文没有直接存明文，而是用 zlib 无损压缩后存储。这样既能降低数据库体积，又不会损失内容；读取详情页时再解压展示。”

8. 美食推荐算法

美食推荐支持：

- 按热度排序
- 按评分排序
- 按距离排序
- 按菜系过滤
- 模糊搜索美食名、饭店名、菜系、窗口名

推荐分数：

- 热度排序：直接使用 popularity
- 评分排序：rating 为主，popularity 为辅
- 距离排序：距离越近分数越高
- 综合排序：热度、评分、距离加权

选择原因：

- 美食推荐和景区推荐一样，用户通常只看前 10 个。
- Top-K 排序能避免完整排序。
- 距离通过 Dijkstra 得到实际道路距离，比直线距离更合理。

复杂度：

Top-K: $O(N \log K)$
如果按距离排序，需要额外一次 Dijkstra: $O((V + E) \log V)$
模糊查找：与候选数量和字符串长度相关

9. AIGC 旅游动画生成

AIGC 动画生成不是传统确定性算法，而是调用图生视频模型服务。系统负责：

- 从日记图片中选择生成素材。
- 提供默认提示词，并允许用户修改。
- 提供横屏/竖屏、480p/720p、时长、采样步数、seed 等参数。
- 将图片和参数发送到 Wan 视频生成接口。
- 接收 mp4 结果，保存到 uploads 目录，并在日记详情页展示历史生成结果。

选择原因：

- 视频生成模型计算量大，不适合在本系统内自行训练或本地实现。
- 通过接口解耦，系统可以专注于旅游日记业务流程和参数管理。
- 用户可以基于同一张照片多次调整提示词和参数，生成不同效果的视频。

答辩说法：

“AIGC 部分采用模型接口调用方式，系统不是简单上传文件，而是把日记图片、提示词、分辨率、帧数、采样步数和 seed 统一封装成请求，生成完成后保存并关联到日记，形成完整的旅游记录流程。”

10. 算法选择总结

需求特点	算法选择	原因
只需要展示前 10 条	Top-K 最小堆	比完整排序更符合推荐场景
名称、类别查询	Trie + 模糊匹配	前缀查找快，编辑距离可容错
长文本日记搜索	BM25F 全文检索	比简单字符串包含更适合正文相关性排序
路线最短距离/时间	Dijkstra	图边权非负，算法稳定
多点游览路线	贪心 + 2-opt	TSP 精确解成本高，近似算法更适合实际系统
附近设施距离排序	Dijkstra 路径距离	满足不能使用直线距离的要求
日记正文存储	zlib 无损压缩	降低存储占用，保证内容不丢失
图片生成视频	AIGC 接口	利用成熟模型能力，系统负责业务集成

11. 五分钟内算法讲解口径

可以这样讲：

“系统里主要用了四类算法。第一类是推荐排序，旅游推荐、日记推荐和美食推荐都使用 Top-K 最小堆，只维护前 K 个结果，复杂度是 $O(N \log K)$ ，比完整排序更适合用户只看前十条的场景。第二类是查询算法，名称和类别查询用 Trie 和编辑距离模糊匹配，日记正文用 BM25F 全文检索。第三类是路径算法，景区、校园和室内导航都抽象成图，单点路线使用 Dijkstra，多点路线先计算关键点距离矩阵，再用贪心和 2-opt 做 TSP 近似优化。第四类是存储和 AIGC，日记正文用 zlib 无损压缩，图片生成视频通过 Wan 图生视频接口完成。整体选择原则是：推荐场景避免完整排序，路线场景基于真实道路图，查询场景按短文本和长文本分别处理，保证功能可演示、性能可解释。”

12. 可能被问到的问题

Q1：为什么不用数据库直接排序？

数据库排序可以完成基础功能，但课程要求强调核心排序算法实现，尤其是前 10 个推荐不经过完全排序。因此系统在业务层计算综合分数，再用 Top-K 最小堆维护前 K 个结果，能体现算法设计。

Q2：为什么路线规划用 Dijkstra？

因为道路距离和时间成本都是非负数，Dijkstra 正适合非负权图的单源最短路径问题。系统还用最小堆优化优先队列，提高了节点较多时的效率。

Q3：多点路线为什么不是绝对最优？

多点路线接近 TSP 问题，精确枚举所有访问顺序的复杂度是阶乘级。系统采用贪心生成初始路线，再用 2-opt 优化，能在较短时间内得到效果较好的近似路线，更适合实际演示和交互。

Q4：附近设施为什么不是直线距离？

系统先在道路图上运行 Dijkstra，得到从当前位置到每个设施节点的最短路径距离，再排序。所以距离是实际可走路径距离，不是地图上的直线距离。

Q5：AIGC 算法是自己实现的吗？

视频生成模型本身通过 Wan 图生视频服务提供，系统实现的是业务集成算法流程：图片选择、提示词管理、参数配置、接口调用、视频保存和历史结果展示。核心旅游系统算法，如排序、查找、路径规划和压缩存储，是在项目中实现或集成完成的。